Campus Store Payments

A store and payments platform: retry-safe card payments, 3-D Secure, refunds under concurrency, signed webhooks and a merchant dashboard.

Saumya Goswami · Technical documentation · October 2026

Live demo: 30-saumya-11.github.io/secure-checkout/ · GitHub: github.com/30-Saumya-11/secure-checkout

Contents

1. Overview

2. Features

3. Architecture

4. Data model

5. Never charging twice

6. Concurrency

7. Card data and the fraud rule

8. Refunds

9. Webhooks

10. Accessibility and the frontend

11. Payment help assistant (RAG)

12. Testing and delivery

13. Limitations

14. Interview questions

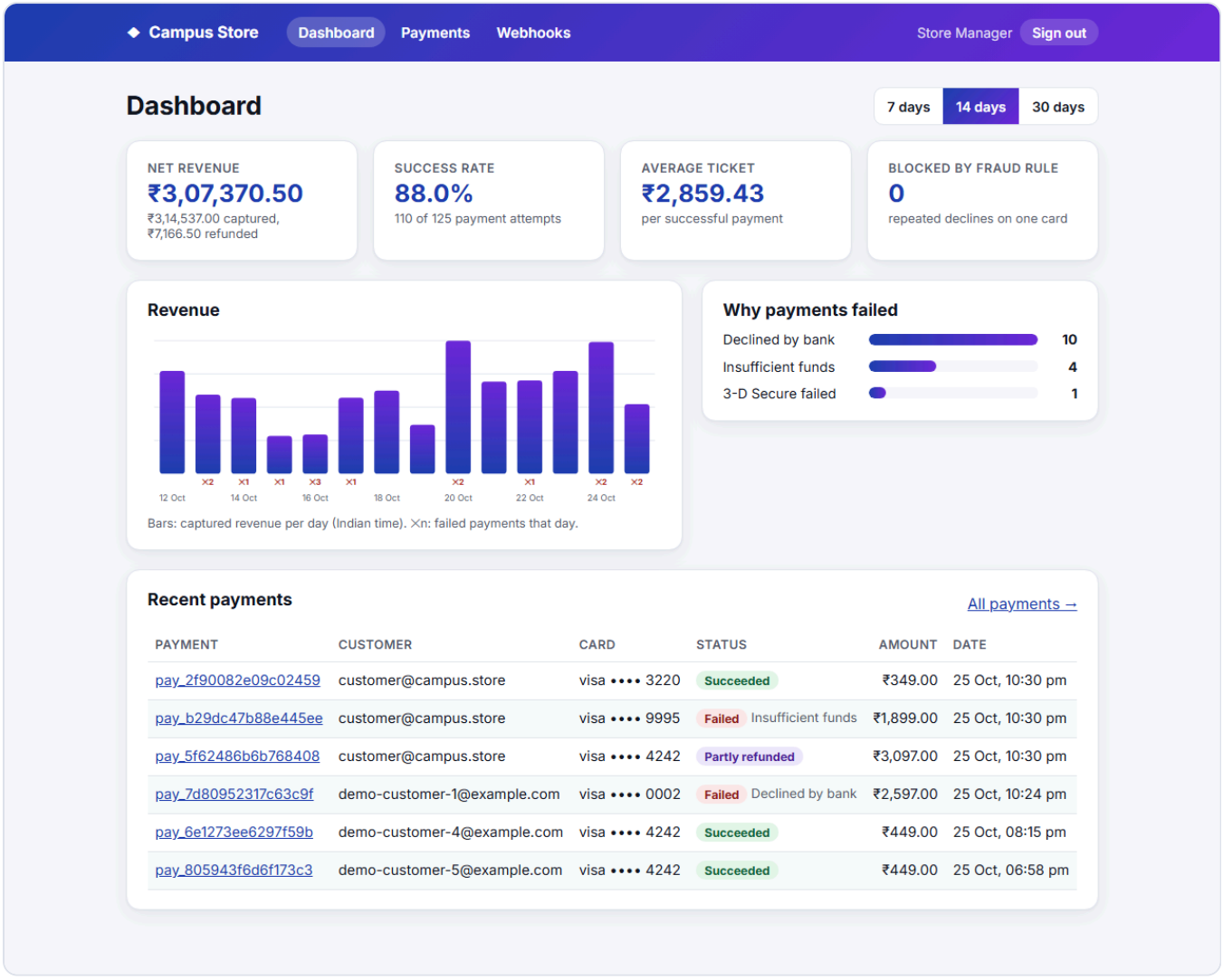
1 Overview

Campus Store Payments is a small online store with its own payments platform. Customers browse products, fill a cart and pay by card. The merchant gets a dashboard with revenue, success rate and decline reasons, can search every transaction, refund all or part of a payment, and watch signed webhooks being delivered to their server.

It is built around the problems that make real payment systems hard: **retries must never charge twice**, **two people acting at once must not break the numbers**, **card data must not be stored**, and **the merchant must be told reliably** about every change.

1 payment from 6 identical requests sent at the same moment	**2 of 5** concurrent ₹1,000 refunds succeed on a ₹2,499 payment, as they must	**109** automated tests (36 backend on Postgres, 73 frontend)	**0** automated accessibility violations on checkout and dashboard

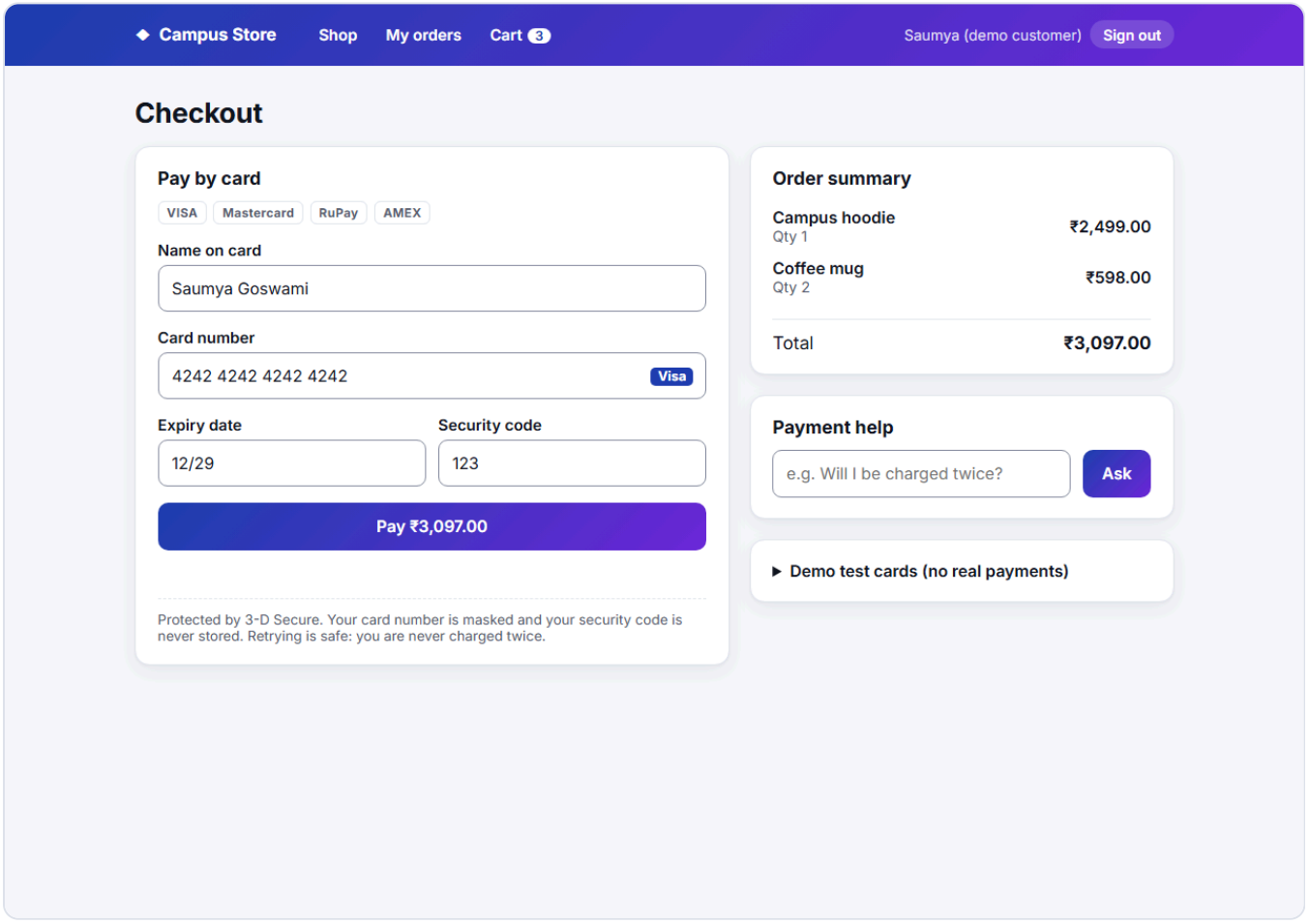
React + TypeScript · FastAPI · PostgreSQL · SQLAlchemy + Alembic · Docker · GitHub Actions · Render blueprint



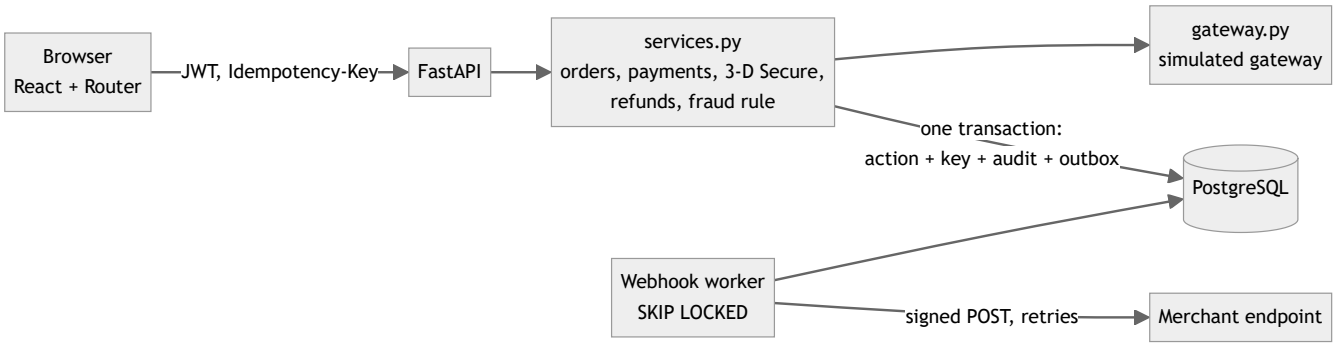
Demo only: the card gateway is simulated and no card network is contacted. The dashboard history is generated by the seed script with test cards and labelled as simulated. Live demo (in-browser demo mode): 30-saumya-11.github.io/secure-checkout

2 Features

Customer	Merchant
Product catalogue and cart (kept between visits)	Dashboard: net revenue, success rate, average ticket, fraud-rule blocks
Sign-in; checkout only for signed-in customers	Revenue per day and failed payments by reason (charts)
Accessible card form with live validation and an error summary	Transactions: search by id, email or last 4 digits; filter by status and card
3-D Secure one-time-code dialog	Payment detail: timeline, customer, items, webhooks
Safe automatic retries after network errors	Full and partial refunds
Order history and receipts	Webhook endpoint, delivery log, manual retry
Payment help assistant (RAG) that cites its sources	Audit log and per-route metrics (API)

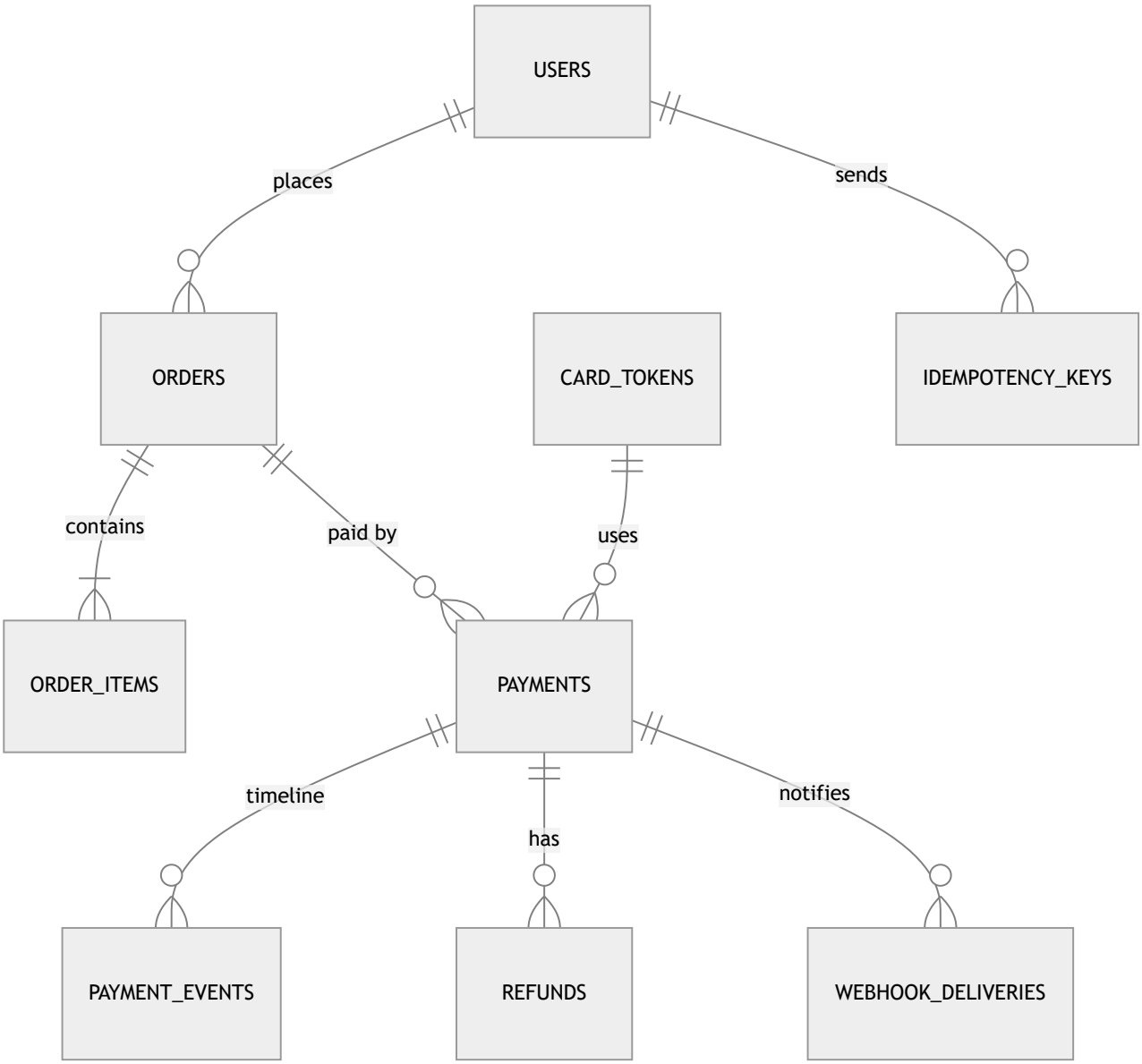


3 Architecture



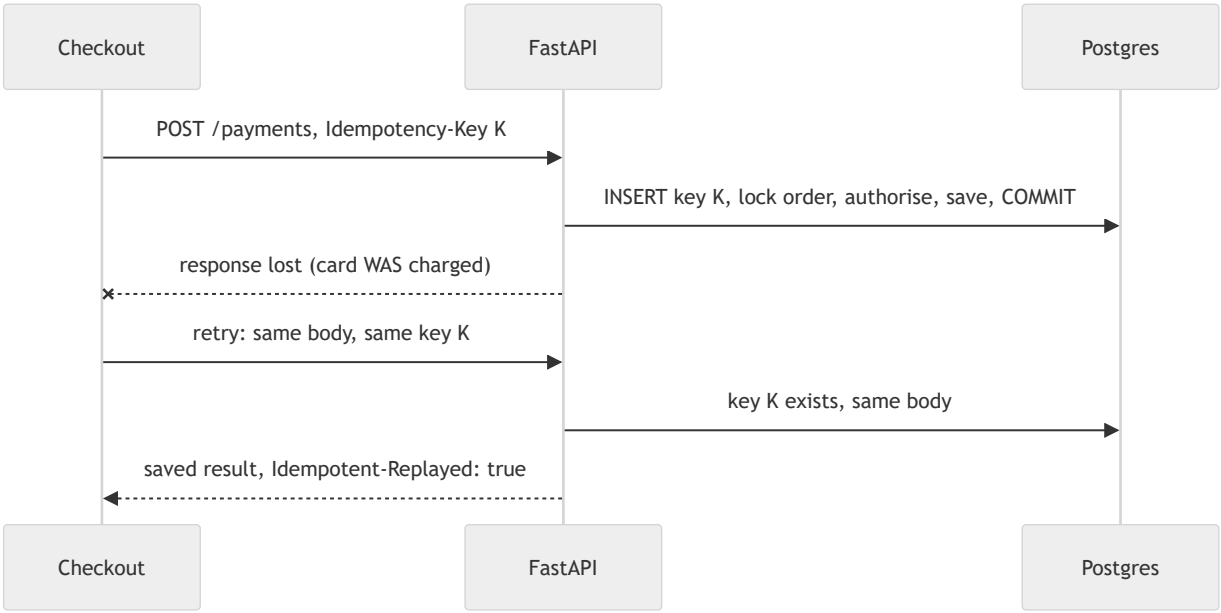
Part	Responsibility
`app/routes/`	HTTP only: validation (Pydantic), roles, status codes
`app/services.py`	Business rules; never commits, so the route commits everything together
`app/idempotency.py`	Stores each key with the action's response, in the same transaction
`app/gateway.py`	Card checks, tokenisation, simulated issuer answers for test cards
`app/webhooks.py`	Signing, verification, delivery worker with back-off
`migrations/`	Alembic migration; the tests build the schema with it, so a broken migration fails CI
`frontend/src/api/`	`HttpApi` for the backend; `DemoApi` runs the same pages in the browser for the static demo

4 Data model



- Money is an integer number of paise everywhere; never a float.
- Public ids are random (`pay_…` , `ord_…`), so nobody can guess other customers' records.
- `card_tokens` holds a token, brand, last 4, expiry and a **keyed fingerprint** (HMAC-SHA256 of the number with a server secret). Never the number, never the CVC.
- A `CHECK` constraint keeps `amount_refunded ≤ amount` even if the code had a bug.

5 Never charging twice



1. **Insert the key first** (`INSERT … ON CONFLICT DO NOTHING`). If another request with the same key is still running, Postgres makes this insert wait until it finishes.
2. **New key** → run the action inside a savepoint, store its response on the key row, and commit once: action, key, audit row and webhook outbox together.
3. **Known key** → same body: replay the stored response. Different body: **409**, because a client bug must not receive someone else's result.
4. An unexpected crash rolls everything back, key included, so the client can simply retry.

The browser creates one key per payment attempt and reuses it (and the same card token) for automatic retries; a new key is used only for a genuinely new attempt. The demo tool `X-Simulate-Lost-Response` commits the payment and then answers 503, to show the retry being replayed.

6 Concurrency

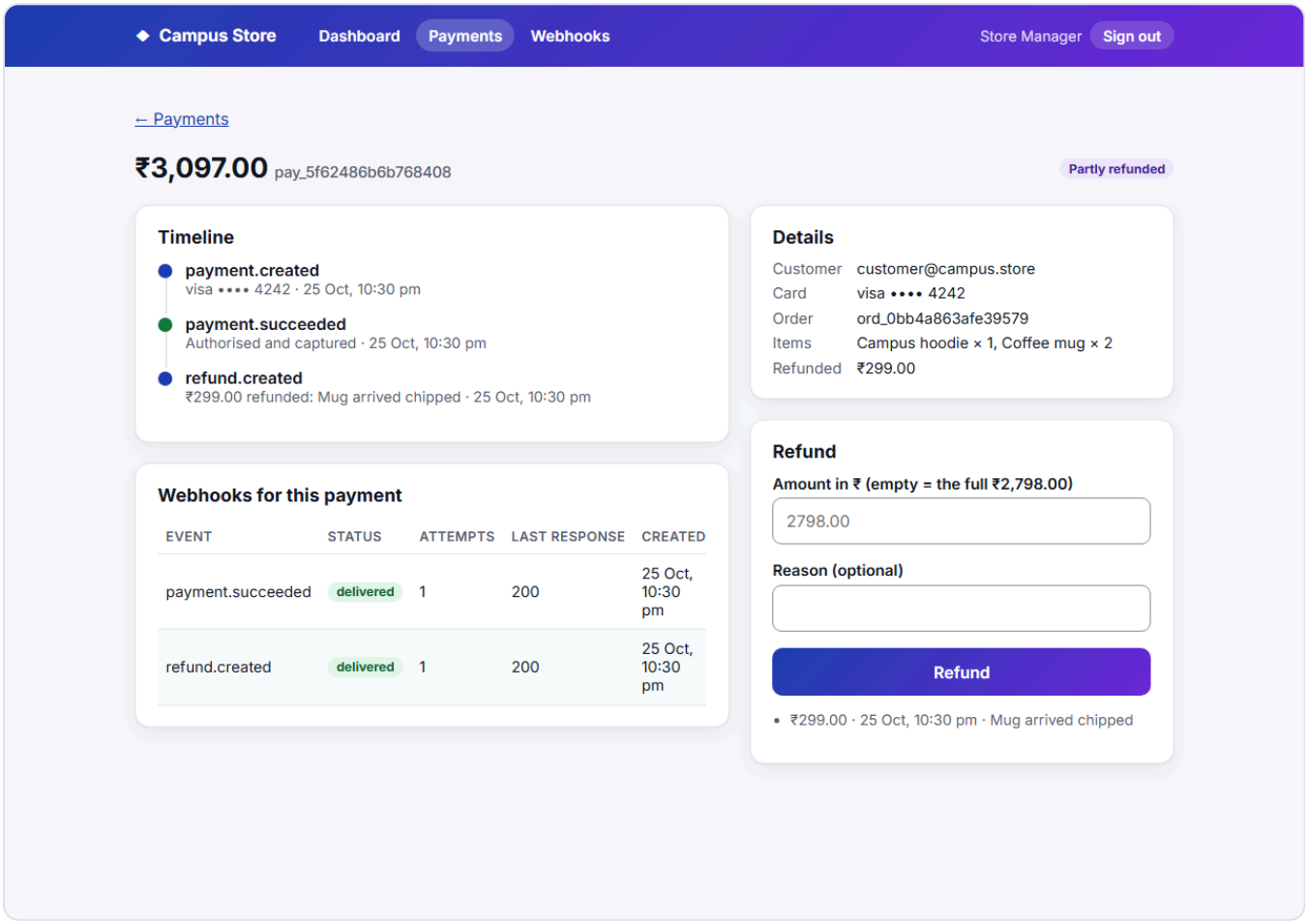
Situation	Protection	Measured in tests
Double click plus automatic retry	Idempotency key row (unique per user)	6 identical requests at once → 1 payment, all 6 get the same answer
Two payments for one order	`SELECT … FOR UPDATE` on the order	5 at once → 1 succeeds, 4 get 409
Two support agents refund at once	`FOR UPDATE` on the payment + `CHECK`	5 × ₹1,000 on ₹2,499 → exactly 2 succeed, ₹2,000 refunded
Customer finishes an abandoned 3-D Secure after paying another way	Order locked at confirmation; old attempt canceled	409, only 1 successful payment
Several webhook workers	`FOR UPDATE SKIP LOCKED`	design; one worker in the demo

7 Card data and the fraud rule

The checkout sends the card to `POST /api/cards/tokens`, which plays the gateway's role: Luhn checksum, brand by IIN range, length, expiry (valid to the end of the month) and CVC length are checked, then the number and CVC are dropped. Only the token is used to pay. A test dumps every table after a payment and searches for the card number.

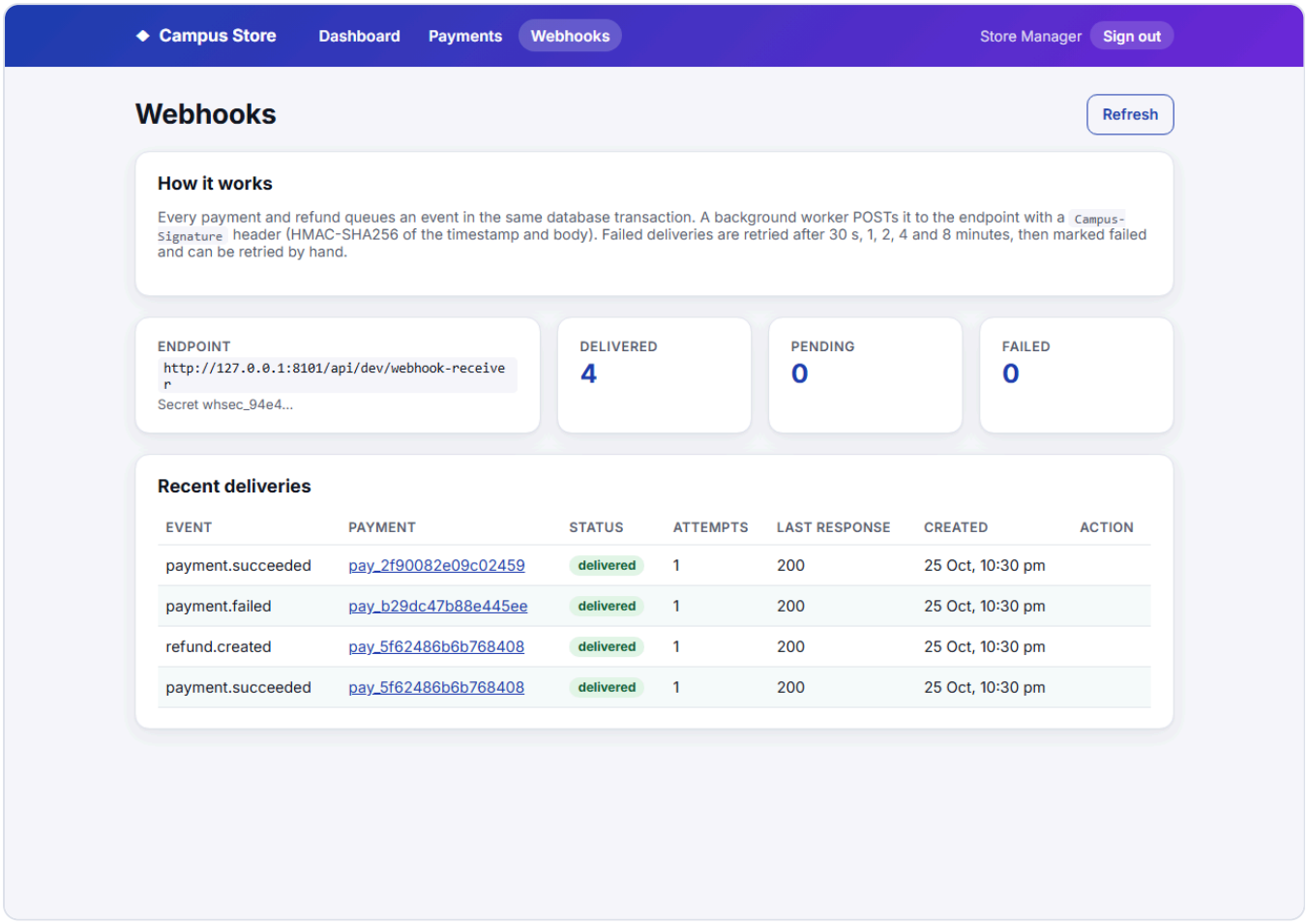
Velocity rule: if the same card (same fingerprint) was declined 3 times in the last 10 minutes, the next attempt is blocked without asking the issuer, recorded with the code `velocity_blocked`, shown on the dashboard and in the payment timeline. Both numbers are settings.

8 Refunds



- Full (leave the amount empty) or partial; the status becomes *partly refunded* or *refunded*, and a full refund marks the order refunded.
- Each refund form submission carries its own idempotency key, so a retried click refunds once.
- Only succeeded or partly refunded payments can be refunded; more than what is left is a 409.
- Every refund adds a timeline event, an audit row and a `refund.created` webhook.

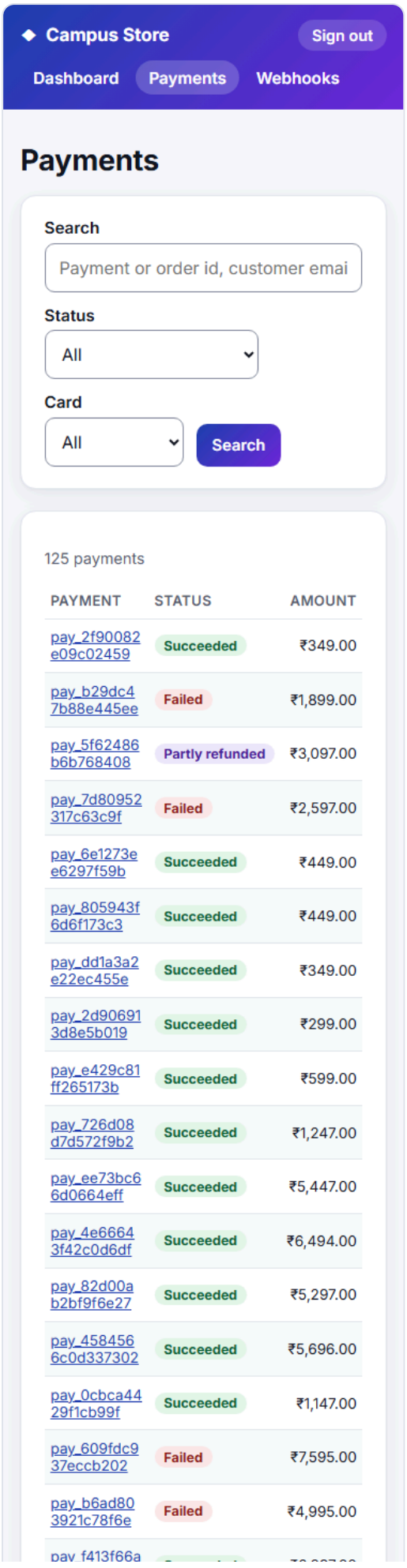
9 Webhooks

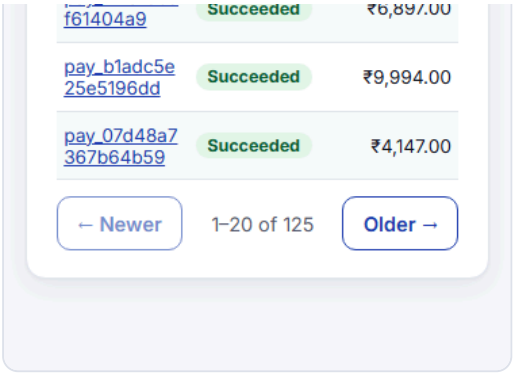


- **Outbox:** the delivery row is written in the same transaction as the payment, so an event is never lost or sent for a change that was rolled back.
- **Signature:** `Campus-Signature: t=<unix time>,v1=<HMAC-SHA256(secret, "t.body")>`. The receiver recomputes it, compares in constant time and rejects timestamps older than 5 minutes (replays).
- **Retries:** after 30 s, 1, 2, 4 and 8 minutes; then *failed*, with a manual retry button. Every retry has the same event id so the merchant can ignore duplicates.
- In demo setups a built-in receiver verifies the signatures, so the log shows real signed deliveries end to end.

10 Accessibility and the frontend

- Labelled fields with the right `autocomplete` tokens, errors linked by `aria-describedby`, an error summary that takes focus, a polite live region for payment progress, keyboard-only flow, a skip link.
- Charts are SVG with text alternatives; status is never shown by colour alone.
- Light theme with AA-contrast status pills; tables drop secondary columns on phones, so no page scrolls sideways at 375 px.
- **Demo mode:** with no API URL configured, the same pages run against an in-browser stand-in (the GitHub Pages demo), with a banner saying so.


Campus Store
Sign out
Dashboard
Payments
Webhooks
Payments
Search
Payment or order id, customer emai
Status
All
Card
All
Search
125 payments
PAYMENT
STATUS
AMOUNT
pay_2f90082e09c02459
Succeeded
₹349.00
pay_b29dc47b88e445ee
Failed
₹1,899.00
pay_5f62486b6b768408
Partly refunded
₹3,097.00
pay_7d80952317c63c9f
Failed
₹2,597.00
pay_6e1273ee6297f59b
Succeeded
₹449.00
pay_805943f6d6f173c3
Succeeded
₹449.00
pay_dd1a3a2e22ec455e
Succeeded
₹349.00
pay_2d906913d8e5b019
Succeeded
₹299.00
pay_e429c81ff265173b
Succeeded
₹599.00
pay_726d08d7d572f9b2
Succeeded
₹1,247.00
pay_ee73bc66d0664eff
Succeeded
₹5,447.00
pay_4e66643f42c0d6df
Succeeded
₹6,494.00
pay_82d00ab2bf9f6e27
Succeeded
₹5,297.00
pay_4584566c0d337302
Succeeded
₹5,696.00
pay_0cbca4429f1cb99f
Succeeded
₹1,147.00
pay_609fdc937eccb202
Failed
₹7,595.00
pay_b6ad803921c78f6e
Failed
₹4,995.00



11 Payment help assistant (RAG)

10 help articles (21 sections), BM25 retrieval with synonyms and light stemming, and the last decline code added as context ("why did it fail?" is answered for *this* payment). Below a score threshold chosen on dev questions only, it says it could not find the answer.

20 held-out test questions	Result
Right article ranked first	14 / 16
Right article in the top 3	15 / 16
Out-of-scope questions refused	3 / 4
In-scope questions wrongly refused	0 / 16

> The articles and questions were written by the same person and the set is small, so these numbers are optimistic.

12 Testing and delivery

Suite	Tests	Covers
backend payments	14	idempotency (incl. concurrent), lost responses, declines, 3-D Secure, fraud rule, no card data stored
backend refunds and auth	12	limits, concurrent refunds, roles, ownership, forged and unsigned tokens
backend webhooks and dashboard	10	signatures, back-off, give-up, dashboard numbers, search, metrics, audit
frontend pages and API client	17	cart, checkout to receipt, role guards, dashboard (axe), filters, refunds, client retries
frontend checkout and assistant	56	card rules, gateway, form accessibility, assistant evaluation floor

Backend tests run on a real PostgreSQL (CI uses a Postgres service container; locally a throwaway server is started), because row locks and `ON CONFLICT` behave differently in SQLite. GitHub Actions runs both suites and builds the API image; a second workflow publishes the demo-mode frontend to GitHub Pages. `docker compose up` runs Postgres, the API and the web app behind nginx; `render.yaml` deploys the same to Render.

13 Limitations

- The gateway is simulated. A real store would use a provider's hosted card fields, so the card number never reaches this server (smaller PCI DSS scope).
- The session token lives in `localStorage`; an `httpOnly` cookie with CSRF protection would be safer.
- Metrics are in-process; one fixed fraud rule, not a model; dashboard history is simulated.

14 Interview questions

Why store the idempotency key in the same transaction as the payment?

If they were separate, a crash between them would leave a charge without a key (the retry charges again) or a key without a charge (the retry is wrongly told it worked).

What happens when two requests insert the same key at once?

The second insert waits on the unique index until the first commits, then finds the row and replays its stored response.

Why lock the payment row for refunds if there is a CHECK constraint?

The lock makes the second refund see the first and get a clear 409; the constraint is a last line of defence that would otherwise surface as a 500.

Why an outbox instead of sending the webhook in the request?

Sending inside the request could notify the merchant about a change that then rolls back, or lose the event if the process dies after commit. The outbox row commits with the change.

Why include a timestamp in the webhook signature?

Without it, anyone who captured one request could replay it forever; with it, the receiver rejects old requests.

Why a keyed fingerprint and not a plain hash of the card number?

Card numbers have little entropy; a plain hash can be reversed by hashing all candidates. The HMAC needs the server secret.